# 実験データ記録ダイアログ UI設計書

## メタデータ

- **画面ID**: DataRecordDialog
- **画面名**: 実験データ記録ダイアログ
- **ファイルパス**: `frontend/src/components/ExperimentWorker/DataRecordDialog.tsx`
- **利用権限**: 実験作業者（experiment_worker）
- **作成日**: 2026-01-21
- **最終更新日**: 2026-03-16

---

## 1. 概要

### 1.1 画面の目的

トランジション（実験工程）の実験データを記録するためのモーダルダイアログです。テンプレートを選択し、各プロパティに実験結果を入力してデータを保存します。

### 1.2 主要機能

- テンプレートの選択（Materials/Conditions/Results）
- プロパティタイプ別のデータ入力
- DataGrid編集（行列の動的追加・削除）
- トランジションステータスの更新
- みなし進捗更新機能
- 作業指示の確認
- プロジェクトステータスに応じた編集制御

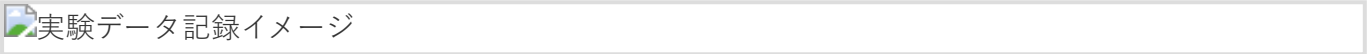
### 1.3 アクセス権限

- **対象ロール**: 実験作業者（experiment_worker）
- **必要条件**: ログイン済み、有効なセッション、編集可能なプロジェクト

---

# 2. 画面構成

## 2.1 画面イメージ




## 2.2 レイアウト構造

```
┌──────────────────────────────────────────────────────────┐
│ Modal (width: 1400px)                                    │
│ ┌──────────────────────────────────────────────────────┐ │
│ │ Title: 実験データ記録 - [トランジション名]             │ │
│ ├──────────────────────────────────────────────────────┤ │
│ │ Body:                                                │ │
│ │ ┌──────────────────────────────────────────────────┐ │ │
│ │ │ □ みなし進捗更新    [作業指示] ボタン             │ │ │
│ │ └──────────────────────────────────────────────────┘ │ │
│ │                                                      │ │
│ │ ┌──────────────────────────────────────────────────┐ │ │
│ │ │ ▼ ステータス更新（みなし進捗OFFの時のみ表示）    │ │ │
│ │ │ ┌──────────┬─────────────────────────────────┐ │ │ │
│ │ │ │ ステータス  │ 進捗スライダー (0%-100%)        │ │ │ │
│ │ │ ├──────────┴─────────────────────────────────┤ │ │ │
│ │ │ │ コメント (TextArea)                        │ │ │ │
│ │ │ ├────────────────────────────────────────────┤ │ │ │
│ │ │ │                          [保存] ボタン     │ │ │ │
│ │ │ └────────────────────────────────────────────┘ │ │ │
│ │ └──────────────────────────────────────────────────┘ │ │
│ │                                                      │ │
│ │ Layout (2カラム, height: 600px)                      │ │
│ │ ┌──────────┬─────────────────────────────────┐ │ │ │
│ │ │ Sider    │ Content                         │ │ │ │
│ │ │ (300px)  │ (残り幅)                        │ │ │ │
│ │ ├──────────┼─────────────────────────────────┤ │ │ │
│ │ │ テンプレート│ データ記録                      │ │ │ │
│ │ │ 一覧     │                                 │ │ │ │
│ │ │          │ ┌─────────────────────────────┐ │ │ │ │
│ │ │ [検索]   │ │ Form (選択されたテンプレート)  │ │ │ │ │
│ │ │ [種別]   │ │ - プロパティ入力フォーム       │ │ │ │ │
```

```
| | |           | | - DataGrid                          | |   | |
| | | ┌─────────┐| | - ネストプロパティ                  | |   | |
| | | | Table   || | - ファイルアップロード              | |   | |
| | | |材料 IN  || └──────────────────────────────────┘ |   | |
| | | |条件 IN  ||                                      |   | |
| | | |結果 OUT || ※未選択時: Alertメッセージ表示       |   | |
| | | └─────────┘|                                      |   | |
| | └────────────┴──────────────────────────────────────┘   | |
| ├──────────────────────────────────────────────────────────┤ |
| | Footer:                                                | |
| | [キャンセル]                        [保存] ボタン      | |
| └──────────────────────────────────────────────────────────┘ |
└──────────────────────────────────────────────────────────────┘


DataGrid編集画面:
┌──────────────────────────────────────────┐
| プロパティ名 [単位]                       |
| ┌──────────────────────────────────────┐ |
| | Table (bordered, size="small")       | |
| | #  | 列1 [x] | 列2 [x] | 列3 [x]     | |
| | 1[x]| 0.00    | 0.00    | 0.00        | |
| | 2[x]| 0.00    | 0.00    | 0.00        | |
| └──────────────────────────────────────┘ |
| [行を追加] [列を追加]                     |
└──────────────────────────────────────────┘

子モーダル:
- InstructionConfirmationModal（作業指示確認）
```

## 2.3 使用コンポーネント

### Ant Design コンポーネント

- `Modal` - ダイアログコンテナ (width: 1400px, top: 20)
- `Layout` - 2カラムレイアウト (Sider + Content)
- `Sider` - 左サイドバー (width: 300px)
- `Content` - 右コンテンツエリア
- `Form` - データ入力フォーム
- `Input` / `TextArea` - テキスト入力
- `InputNumber` - 数値入力
- `Checkbox` - チェックボックス

- `DatePicker` - 日付/日時選択
- `Select` - ドロップダウン選択（ステータス、テンプレート種別）
- `Table` - テンプレート一覧、DataGrid
- `Button` - 各種ボタン
- `Space` - 要素の間隔制御
- `Row` / `Col` - グリッドレイアウト
- `Alert` - 情報メッセージ
- `Tag` - ラベル表示
- `Upload` - ファイルアップロード
- `Slider` - 進捗スライダー
- `Typography.Title` - 見出し
- `Typography.Text` - テキスト表示
- `theme` - デザイントークン取得

### カスタムコンポーネント

- `InstructionConfirmationModal` - 作業指示確認モーダル

### アイコン

- `PlusOutlined` - 追加アイコン（行列追加）
- `DeleteOutlined` - 削除アイコン（行列削除）
- `UploadOutlined` - アップロードアイコン
- `SaveOutlined` - 保存アイコン
- `CloseOutlined` - 閉じるアイコン
- `FileTextOutlined` - ファイルアイコン
- `DownOutlined` - 下向き矢印（展開）
- `RightOutlined` - 右向き矢印（折りたたみ）
- `SearchOutlined` - 検索アイコン
- `AppstoreOutlined` - アプリアイコン（テンプレート一覧）

- `InfoCircleOutlined` - 情報アイコン（作業指示）

---

# 3. コントロール一覧

## 3.1 入力項目

### 上部コントロール

| 項目名 | タイプ | 必須 | 初期値 | 表示条件 | 説明 |
|---|---|---|---|---|---|
| useDeemedProgress | Checkbox | - | Props.useDeemedProgress | useDeemedProgressPropがtrueの場合のみ表示 | みなし進捗更新フラグ（ON: ステータス更新非表示、OFF: 表示） |

> *注意: Propsの `useDeemedProgress` が `false`（組織設定でみなし進捗を使用しない）の場合、チェックボックス自体を非表示にしてステータス更新セクションを常時表示します。*

### ステータス更新セクション

| 項目名 | タイプ | 必須 | 初期値 | 説明 |
|---|---|---|---|---|
| selectedStatus | Select | ○ | 遷移可能な最初のステータス（数値） | トランジションステータス（ALLOWED_TRANSITIONSによる遷移制限あり） |
| progressRate | Slider | - | 0 | 進捗率（0-100%） |
| comment | TextArea | - | '' | コメント |

**表示条件:**

- `useDeemedProgressProp === false`: 常時表示
- `useDeemedProgressProp === true` かつ `useDeemedProgress（内部State） === false`: 表示
- `useDeemedProgressProp === true` かつ `useDeemedProgress（内部State） === true`: 非表示

- 現在のステータスが終端（5 or 6）の場合: `selectedStatus` を disabled にして変更不可の警告 Alert を表示

## テンプレート一覧フィルタ

| 項目名 | タイプ | 必須 | 初期値 | 説明 |
| --- | --- | --- | --- | --- |
| templateSearchText | Input | - | '' | テンプレート名検索 |
| selectedTemplateCategory | Select | - | undefined | テンプレート種別（Materials/Conditions/Results） |

## データ入力フォーム（選択されたテンプレートに依存）

| プロパティタイプ | コンポーネント | 説明 |
| --- | --- | --- |
| stringType | TextArea | 文字列入力（3行） |
| intType | InputNumber | 整数入力（step: 1, precision: 0） |
| floatType | InputNumber | 小数入力（step/precision は `formatString` から算出。`formatString` 未指定時は step: 0.01, precision: 2） |
| booleanType | Checkbox | 真偽値 |
| dateType | DatePicker | 日付（YYYY-MM-DD） |
| dateTimeType | DatePicker | 日時（YYYY-MM-DD HH:mm:ss） |
| contentDoubleListType | DataGrid | 2次元数値配列 |
| propertyListType | ネストフォーム | 階層化されたプロパティ |
| insertion | Upload | ファイルアップロード |

**プロパティラベルの表示:**

- `units` が設定されている場合: ラベル右に青色タグで単位を表示
- `scaleFactor` が設定されている場合: ラベル右に紫色タグで「×{scaleFactor}」を表示
- `description` が設定されている場合: 入力コンポーネントの下部（Form.Item の `extra`）に薄グレーの説明文を表示

## 3.2 ボタン

| ボタン名 | タイプ | 表示条件 | アクション |
|---|---|---|---|
| 作業指示 | default + InfoCircleOutlined | 常時 | InstructionConfirmationModalを開く |
| ステータス更新保存 | primary + SaveOutlined | ステータス更新セクション表示時 & プロジェクト編集可能 | ステータスを保存（モック） |
| 行を追加 | small + PlusOutlined | DataGrid表示時 & プロジェクト編集可能 | DataGridに行を追加 |
| 列を追加 | small + PlusOutlined | DataGrid表示時 & プロジェクト編集可能 | DataGridに列を追加 |
| 行削除 | text + danger + DeleteOutlined | DataGrid各行 & プロジェクト編集可能 | 行を削除（最低1行は残す） |
| 列削除 | text + danger + DeleteOutlined | DataGrid各列 & プロジェクト編集可能 | 列を削除（最低1列は残す） |
| キャンセル | default | 常時 | ダイアログを閉じる |
| 保存 | primary + SaveOutlined | 常時 | テンプレートデータを保存 |

## 3.3 表示項目

### テンプレート一覧テーブル

| カラム | 内容 | 説明 |
|---|---|---|
| テンプレート名 | Text（太字） | テンプレートの名前 |
| プロパティ数 | Text（副次） | (X項目) |
| テンプレート種別タグ | Tag | 材料（青）/ 条件（緑）/ 結果（橙） |
| I/Oタグ | Tag | IN（青）/ OUT（緑） |
| 記録済みバッジ | Badge（緑ドット） | `fetchedTemplateItems` に該当 templateId のデータが存在する場合に表示 |

選択状態: `rowClassName` でハイライト表示

> ***注記:*** *`fetchedTemplateItems`（GET API で取得した既存データ）に一致する `templateId` が存在する行には、記録済みを示す緑ドットバッジを表示します。*

## ステータス選択肢

`TransitionStatus = number` 型（数値）で管理します。

| 値（数値） | ラベル | 備考 |
|---|---|---|
| 0 | 初期状態 | 初期値 |
| 1 | 準備状態 | |
| 2 | 準備完了状態 | |
| 3 | 動作状態 | |
| 4 | 休止状態 | |
| 5 | 終状態 | **終端ステータス（変更不可）** |
| 6 | 異常終了状態 | **終端ステータス（変更不可）** |

> ***注意**: 終端ステータス（5: 終状態、6: 異常終了状態）の場合は、ステータス変更不可として警告 Alert を表示します。*

## ステータス遷移ルール

現在のステータスから遷移可能なステータスのみ Select の選択肢に表示します（`ALLOWED_TRANSITIONS` に基づく）。

| 現在のステータス | 遷移可能なステータス |
|---|---|
| 0: 初期状態 | 1: 準備状態、5: 終状態、6: 異常終了状態 |
| 1: 準備状態 | 2: 準備完了状態、5: 終状態、6: 異常終了状態 |
| 2: 準備完了状態 | 2: 準備完了状態（自己遷移）、3: 動作状態、5: 終状態、6: 異常終了状態 |
| 3: 動作状態 | 4: 休止状態、5: 終状態、6: 異常終了状態 |
| 4: 休止状態 | 3: 動作状態、5: 終状態、6: 異常終了状態 |
| 5: 終状態 | **遷移不可（変更不可）** |
| 6: 異常終了状態 | **遷移不可（変更不可）** |

- 現在のステータスは `transition.status`（数値）から取得（APIレスポンスでは数値が格納されるため `Number()` で変換）
- `selectedStatus` の初期値はダイアログオープン時に「現在のステータスから遷移可能な最初の選択肢」を自動設定

- 終端ステータス（5/6）の場合は `statusOptions` が空配列になり、警告 Alert を表示して操作を無効化

### 空状態メッセージ

| 状態 | メッセージ | タイプ |
| --- | --- | --- |
| テンプレート未選択 | テンプレートを選択してください<br>左側のテンプレート一覧から記録したいテンプレートを選択してください。 | info |

---

# 4. イベント

## 4.1 ダイアログイベント

| イベント名 | トリガー | 処理内容 |
| --- | --- | --- |
| ダイアログ表示 | open prop変更 | フォームリセット、初期値設定、DataGrid初期化 |
| onClose | キャンセルボタン / 閉じるボタン | ダイアログを閉じる |

## 4.2 テンプレート選択イベント

| イベント名 | トリガー | 処理内容 |
| --- | --- | --- |
| onRow.onClick | テンプレート行クリック | selectedTemplateIdを更新、フォーム表示 |
| templateSearchText変更 | 検索入力 | テンプレート一覧をフィルタリング |
| selectedTemplateCategory変更 | 種別選択 | テンプレート一覧をフィルタリング |

## 4.3 データ入力イベント

| イベント名 | トリガー | 処理内容 |
| --- | --- | --- |
| handleSaveTemplate | 保存ボタンクリック | フォームバリデーション → データ整形 → onSave呼び出し |
| handleDataGridChange | DataGridセル入力 | dataGrids State更新 |
| handleAddRow | 行追加ボタン | DataGridに新規行追加 |
| handleAddColumn | 列追加ボタン | DataGridに新規列追加 |

| | | |
|---|---|---|
| handleDeleteRow | 行削除ボタン | 指定行を削除 |
| handleDeleteColumn | 列削除ボタン | 指定列を削除 |

## 4.4 ステータス更新イベント

| イベント名 | トリガー | 処理内容 |
|---|---|---|
| useDeemedProgress変更 | チェックボックス | ステータス更新セクションの表示切替 |
| selectedStatus変更 | ステータス選択 | selectedStatus State更新 |
| progress変更 | スライダー移動 | progress State更新 |
| comment変更 | コメント入力 | comment State更新 |
| ステータス更新保存 | 保存ボタンクリック | ステータス更新データをログ出力（モック） |

## 4.5 作業指示確認イベント

| イベント名 | トリガー | 処理内容 |
|---|---|---|
| 作業指示ボタンクリック | ボタンクリック | InstructionConfirmationModalを開く |
| InstructionConfirmationModal閉じる | モーダルキャンセル | instructionModalOpenをfalseに設定 |

---

# 5. データフロー

## 5.1 Props（親から受け取るデータ）

```
interface DataRecordDialogProps {
  open: boolean;                          // ダイアログの表示状態
  transition: Transition | null;          // 対象トランジション
  projectStatus: ProjectStatus | null;    // プロジェクトステータス
  onClose: () => void;                    // 閉じるハンドラ
  onSave: (data: ExperimentDataValue[]) => void; // 保存ハンドラ
  projectId?: string;                     // プロジェクトID（作業指示用）
  projectName?: string;                   // プロジェクト名（実験データ保存API・テンプレート実績取得APIのパスパラメータ）
  methodId?: string;                      // メソッドID（作業指示用・実験データ保存APIのパスパラメータ）
  taskNo?: number;                        // タスク連番（実験データ保存APIのパスパラメータ）
```

```
  /** みなし進捗使用フラグ（organization.use_deemed_progress）: false の場合はチェックボックスを非表示にしてステータス更新を常時表示 */
  useDeemedProgress?: boolean;            // 組織設定のみなし進捗フラグ（省略時: true）
}
```

## 5.2 State（内部状態）

```
// フォームインスタンス
const [form] = Form.useForm();

// フォームデータ
const [formData, setFormData] = useState<Record<string, ExperimentDataValue>>({});

// DataGrid管理
const [dataGrids, setDataGrids] = useState<Record<string, number[][]>>({});

// ローディング状態（テンプレート保存・ステータス更新共用）
const [loading, setLoading] = useState(false);

// テンプレート実績データのローディング状態（ダイアログオープン時のGET API用）
const [templateDataLoading, setTemplateDataLoading] = useState(false);

// クライアント判定
const [isClient, setIsClient] = useState(false);

// 作業指示モーダル
const [instructionModalOpen, setInstructionModalOpen] = useState(false);

// APIから取得したテンプレート一覧（TemplateDataItem → Template に変換済み）
// ダイアログオープン時にGET APIで取得。空の場合は transition.templates にフォールバック
const [fetchedTemplates, setFetchedTemplates] = useState<Template[]>([]);

// APIから取得したテンプレート実績の生データ（placeId・data構造の保持に使用）
const [fetchedTemplateItems, setFetchedTemplateItems] = useState<TemplateDataItem[]>([]);

// テンプレート一覧フィルタ
const [templateSearchText, setTemplateSearchText] = useState('');
const [selectedTemplateCategory, setSelectedTemplateCategory] = useState<TemplateCategory |
undefined>(undefined);
const [templatesExpanded, setTemplatesExpanded] = useState(true);

// テンプレート選択（単一選択）
const [selectedTemplateId, setSelectedTemplateId] = useState<string | null>(null);

// プロパティ入力フォーム折りたたみ
const [propertyFormExpanded, setPropertyFormExpanded] = useState(true);
```

```
// ステータス更新折りたたみ
// プロジェクトが編集不可の場合は初期値 false（折りたたみ状態）
const [statusUpdateExpanded, setStatusUpdateExpanded] = useState(true);

// ステータスとコメント
// selectedStatus: TransitionStatus（= number）型。初期値は遷移可能な最初の選択肢
（ALLOWED_TRANSITIONS参照）
const [selectedStatus, setSelectedStatus] = useState<TransitionStatus>(0);
const [comment, setComment] = useState('');
const [progress, setProgress] = useState<number>(0);

// みなし進捗更新
// useDeemedProgressProp（Props）が false の場合はチェックボックス非表示・ステータス更新常時表示
// ダイアログオープン時に Props の最新値を同期
const [useDeemedProgress, setUseDeemedProgress] = useState(useDeemedProgressProp);
```

## 5.3 算出データ（useMemo）

```
// プロジェクト編集可能フラグ
const isProjectEditable = projectStatus ? isEditableStatus(projectStatus) : true;

// 現在のトランジションステータス（数値）
// transition.status は型上 ProgressStatus（文字列型）だが、APIレスポンスでは実際に数値（0-6）が入
るため Number() で変換
const currentStatus = useMemo((): number => {
  if (!transition || transition.status === undefined || transition.status === null) return 0;
  const num = Number(transition.status);
  return isNaN(num) ? 0 : num;
}, [transition]);

// 現在のステータスから遷移可能なステータス選択肢（ALLOWED_TRANSITIONS に基づく）
const statusOptions = useMemo(() => {
  const allowedValues = ALLOWED_TRANSITIONS[currentStatus] ?? [];
  return ALL_STATUS_OPTIONS.filter(opt => allowedValues.includes(opt.value as number));
}, [currentStatus]);

// フィルタ済みテンプレート一覧
// fetchedTemplates（APIから取得・変換済み）を優先し、空の場合は transition.templates にフォールバ
ック
const filteredTemplates = useMemo(() => {
  const source: Template[] =
    fetchedTemplates.length > 0
      ? fetchedTemplates
      : (transition?.templates ?? []);
```

```
  let filtered = source;

  // テンプレート名でのあいまい検索
  if (templateSearchText) {
    filtered = filtered.filter((template) =>
      template.name.toLowerCase().includes(templateSearchText.toLowerCase())
    );
  }

  // テンプレート種別での絞り込み
  if (selectedTemplateCategory) {
    filtered = filtered.filter((template) =>
      template.category === selectedTemplateCategory
    );
  }

  return filtered;
}, [fetchedTemplates, transition?.templates, templateSearchText, selectedTemplateCategory]);

// 選択されたテンプレート
const selectedTemplate = useMemo(() => {
  if (!selectedTemplateId) return null;
  return filteredTemplates.find(template =>
    template.templateId === selectedTemplateId
  );
}, [filteredTemplates, selectedTemplateId]);
```

## 5.4 使用するAPI

| API | メソッド | エンドポイント | 用途 | 実装状況 |
|---|---|---|---|---|
| テンプレート実績取得API | GET | `/api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/template-data` | ダイアログオープン時にテンプレート実績を取得 | 実装済み |
| 実験データ保存API | POST | `/api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/template-data` | テンプレートデータを保存 | 実装済み |
| ステータス更新API | PUT | `/api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/transitions/:transitionId/status` | トランジションステータスの更新 | 実装済み |
| ファイルアップロードAPI | POST | `/api/v1/files/upload` | 実験データファイルのアップロード | TODO |

> ***注意**: パスパラメータは `projectId`（UUID）ではなく `projectName`（文字列）を使用します。*

---

**テンプレート実績取得API詳細（GET）:**

パスパラメータ:

- `projectName`: プロジェクト名（文字列）
- `methodId`: メソッドID（文字列）
- `taskNo`: タスク連番（整数）

レスポンス例:

```
{
  "success": true,
  "data": [
    {
      "id": "template-data-uuid",
      "projectName": "sample_project",
      "methodId": "method_001",
      "taskNo": 1,
      "placeId": "place_001",
      "templateId": "mat_001",
      "templateType": "material",
      "data": { "property1": "value1", "temperature": 25.5 },
      "status": 3,
      "comment": "記録済み",
      "createdAt": "2026-01-01T00:00:00Z",
      "updatedAt": "2026-01-01T00:00:00Z"
    }
  ]
}
```

ダイアログオープン時に取得し、`fetchedTemplates` / `fetchedTemplateItems` State に格納します。

---

**実験データ保存API詳細（POST）:**

パスパラメータ:

- `projectName`: プロジェクト名（文字列）
- `methodId`: メソッドID（文字列）

- `taskNo`: タスク連番（整数）

リクエストボディ:

```
{
  "placeId": "place_001",
  "templateId": "mat_001",
  "templateType": "material",
  "data": {
    "property1": "value1",
    "property2": 123.45
  },
  "status": 3,
  "comment": "材料データ記録完了"
}
```

レスポンス例:

```
{
  "success": true,
  "data": {
    "id": "template-data-uuid",
    "projectName": "sample_project",
    "methodId": "method_001",
    "taskNo": 1,
    "placeId": "place_001",
    "templateId": "mat_001",
    "templateType": "material",
    "data": { "property1": "value1", "property2": 123.45 },
    "status": 3,
    "comment": "材料データ記録完了"
  }
}
```

**注記:**

- 1リクエストにつき1テンプレート分のデータを保存（単一オブジェクト形式）
- `placeId` はテンプレートに紐づく Place の ID（`transition.templates` の `placeId` から取得）
- `templateType` は `material` / `condition` / `result` のいずれか（テンプレートの `category` に対応）

- `status` は数値型（`selectedStatus` の値）
- 既存データがある場合は上書き保存（upsert）

---

**ステータス更新API詳細（PUT）:**

パスパラメータ:

- `projectName`: プロジェクト名（文字列）
- `methodId`: メソッドID（文字列）
- `taskNo`: タスク連番（整数）
- `transitionId`: トランジションID（文字列）

リクエストボディ:

```
{
  "status": 3,
  "progressRate": 50,
  "comment": "作業中"
}
```

レスポンス例:

```
{
  "success": true,
  "data": {
    "transitionId": "transition_001",
    "status": 3,
    "progressRate": 50,
    "comment": "作業中",
    "updatedAt": "2026-01-01T00:00:00Z"
  }
}
```

## 5.5 データソース

- `fetchedTemplates` - GET APIで取得・変換したテンプレート一覧（優先）

- `transition.templates`
  - トランジションに紐づくテンプレート一覧（フォールバック）
- `fetchedTemplateItems` - GET APIで取得した生データ（placeId・data構造の保持用）

---

# 6. エラーチェック

## 6.1 バリデーション

| 項目 | ルール | エラーメッセージ |
| --- | --- | --- |
| 必須プロパティ | required: true | 「{プロパティ名}は必須項目です」 |
| 整数値 | type: intType | 「整数を入力してください」 |
| 小数値 | type: floatType | 「数値を入力してください」 |
| 日付 | type: dateType | 「日付を選択してください」 |
| 最小値 | min設定あり | 「{min}以上の値を入力してください」 |
| 最大値 | max設定あり | 「{max}以下の値を入力してください」 |

## 6.2 保存時のチェック

```
const handleSaveTemplate = async () => {
  if (!selectedTemplate) {
    message.error('テンプレートが選択されていません');
    return;
  }
  if (!projectName || !methodId || taskNo === undefined) {
    message.error('プロジェクト情報が不足しています');
    return;
  }

  try {
    setLoading(true);
    const values = await form.validateFields(); // バリデーション

    // プロパティごとのデータを整形
    const data: Record<string, unknown> = {};
    selectedTemplate.properties.forEach((prop) => {
      const key = `${selectedTemplate.templateId}_${prop.key}`;
```

```
      if (prop.type === 'contentDoubleListType') {
        data[prop.key] = dataGrids[key] || [[0, 0, 0]];
      } else if (prop.type === 'dateType' || prop.type === 'dateTimeType') {
        const dateValue = values[key];
        data[prop.key] = dateValue
          ? dayjs(dateValue).format(
              prop.type === 'dateType' ? 'YYYY-MM-DD' : 'YYYY-MM-DD HH:mm:ss'
            )
          : null;
      } else {
        data[prop.key] = values[key] ?? null;
      }
    });

    // placeId はテンプレートに紐づく Place から取得
    // fetchedTemplateItems に既存データがあれば placeId を再利用
    const existingItem = fetchedTemplateItems.find(
      (item) => item.templateId === selectedTemplate.templateId
    );
    const placeId = existingItem?.placeId ?? selectedTemplate.placeId ?? '';

    // POST API でテンプレートデータを保存
    await fetch(

`/api/v1/experiments/projects/${projectName}/methods/${methodId}/tasks/${taskNo}/template-
data`,
      {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          placeId,
          templateId: selectedTemplate.templateId,
          templateType: selectedTemplate.category,
          data,
          status: selectedStatus,
          comment,
        }),
      }
    );

    onSave([{ templateId: selectedTemplate.templateId, category: selectedTemplate.category,
...data }]);
    message.success(`「${selectedTemplate.name}」を保存しました`);
  } catch (error) {
    console.error('Save failed:', error);
    message.error('保存に失敗しました。入力内容を確認してください');
  } finally {
    setLoading(false);
  }
};
```

## 6.3 編集制御

```
// プロジェクトステータスによる編集可否
const isProjectEditable = projectStatus ? isEditableStatus(projectStatus) : true;

// 全ての入力コンポーネントでdisabledプロパティを設定
disabled={!isProjectEditable}
```

---

# 7. 画面遷移

## 7.1 遷移元

- ExperimentRecordTab（実験記録タブ）から「データ記録」ボタンクリック

## 7.2 遷移先

なし（モーダルダイアログのため）

## 7.3 モーダル遷移

```
DataRecordDialog
  ↳ InstructionConfirmationModal（作業指示確認）
      ↳ 閉じる
```

---

# 8. レスポンシブ対応

## 8.1 ブレークポイント

| サイズ | 幅 | 対応 |
|---|---|---|

| | | |
|---|---|---|
| Mobile | < 768px | 非推奨（モーダル幅1400pxのため） |
| Tablet | 768px - 1024px | 部分対応（横スクロール） |
| Desktop | > 1024px | 完全対応 |

### 8.2 レスポンシブ制御

- モーダル幅: 1400px固定（デスクトップ向け）
- サイドバー幅: 300px固定
- コンテンツエリア: 残り幅を使用
- フォームカラム: 通常12span、DataGrid/ネストプロパティは24span

---

## 9. パフォーマンス考慮事項

### 9.1 最適化

- `filteredTemplates`, `selectedTemplate` は `useMemo` でメモ化
- DataGrid更新は個別セルのみ更新
- 大量のプロパティがある場合の仮想スクロール検討（将来）

### 9.2 データ量

- テンプレート数: 最大100件程度を想定
- DataGrid: 最大100行×20列程度を想定
- プロパティ数: テンプレートあたり最大50項目程度を想定

---

## 10. 備考

### 10.1 特記事項

1. **テンプレート選択方式**

- 単一選択のみ
- 選択したテンプレートのみフォーム表示
- 複数テンプレートの同時記録は非対応（Phase 1）

2. **DataGrid編集**
   - 初期値: 1行3列（[[0, 0, 0]]）
   - 最低1行1列は残す（全削除不可）
   - 小数第2位まで入力可能

3. **ステータス更新機能**
   - みなし進捗更新ON: ステータス更新セクション非表示、保存時に自動判定
   - みなし進捗更新OFF: ステータス更新セクション表示、手動入力
   - プロジェクトが編集不可の場合: ステータス更新セクション初期折りたたみ

4. **プロジェクトステータス制御**
   - `isEditableStatus()` で編集可否を判定
   - 編集不可の場合: 全入力コンポーネントがdisabled

5. **ファイルアップロード**
   - 現在はモック実装（S3アップロード未実装）
   - beforeUploadでアップロード処理を実装予定

## 10.2 将来の拡張

1. 複数テンプレートの同時記録
2. テンプレートのバージョン管理
3. 過去の実験データの参照・コピー機能
4. DataGridのCSVインポート/エクスポート
5. プロパティの条件分岐表示
6. 実験データの差分比較
7. リアルタイムバリデーション
8. オートセーブ機能

## 10.3 関連ドキュメント

- InstructionConfirmationModal UI設計書
- ExperimentRecordTab UI設計書
- 画面一覧
- API一覧（実験作業者画面）

---

# 11. 変更履歴

| バージョン | 日付 | 変更内容 | 担当者 |
|---|---|---|---|
| 1.3.0 | 2026-03-16 | maimlDataWriter仕様修正に伴うUI仕様更新 | Cline |
| | | - `floatType` の precision を固定値（2）から `formatString` 依存に変更（`%.Nf` 形式からN桁を算出） | |
| | | - プロパティラベルに `scaleFactor` タグ表示（紫色、×{scaleFactor}）を追加 | |
| | | - プロパティの `description` を Form.Item の `extra` として説明文表示する仕様を追加 | |
| | | - `Property` 型に `description`・`formatString`・`scaleFactor` フィールドが追加されたことを反映 | |
| 1.2.0 | 2026-03-11 | ソースコード調査に基づく仕様書修正 | Cline |

\|\|\| - Props に `projectName`・`taskNo`・`useDeemedProgress` を追加 \|\|\|\|\| - `TransitionStatus` 型を文字列型→数値型（0〜6）に修正 \|\|\|\|\| - ステータス遷移ルール（`ALLOWED_TRANSITIONS`）を追加 \|\|\|\|\| - State に `templateDataLoading`・`fetchedTemplates`・`fetchedTemplateItems` を追加 \|\|\|\|\| - `selectedStatus` 初期値を数値 `0` に修正 \|\|\|\|\| - `useDeemedProgress` 初期値を Props から同期するよう修正 \|\|\|\|\| - `currentStatus`・`statusOptions` useMemo を追加 \|\|\|\|\| - `filteredTemplates` の data source を `fetchedTemplates` 優先に修正 \|\|\|\|\| - APIエンドポイントを `projectId` → `projectName` に修正 \|\|\|\|\| - テンプレート実績取得API（GET）を追加（実装済み） \|\|\|\|\| - 実験データ保存API・ステータス更新APIを実装済みに更新 \|\|\|\|\| - 保存処理（`handleSaveTemplate`）を API 呼び出し実装に更新 \|\|\|\|\| - みなし進捗チェックボックスの表示条件を詳細化 \|\|\|\|\| - テンプレート一覧に記録済みバッジ列を追加 \|\|\| 1.1.0 | 2026-02-17 | API仕様v2.1.0に準拠した修正 | Cline \|\|\|\| - APIエ

ンドポイント更新（パスパラメータ化） | | | | | - リクエストボディをrecords配列形式に変更 | | | | | - パラメータ名統一（progress → progressRate） | | | | | - placeId必須化の注記追加 | | | 1.0.0 | 2026-01-21 | 初版作成 | Cline |